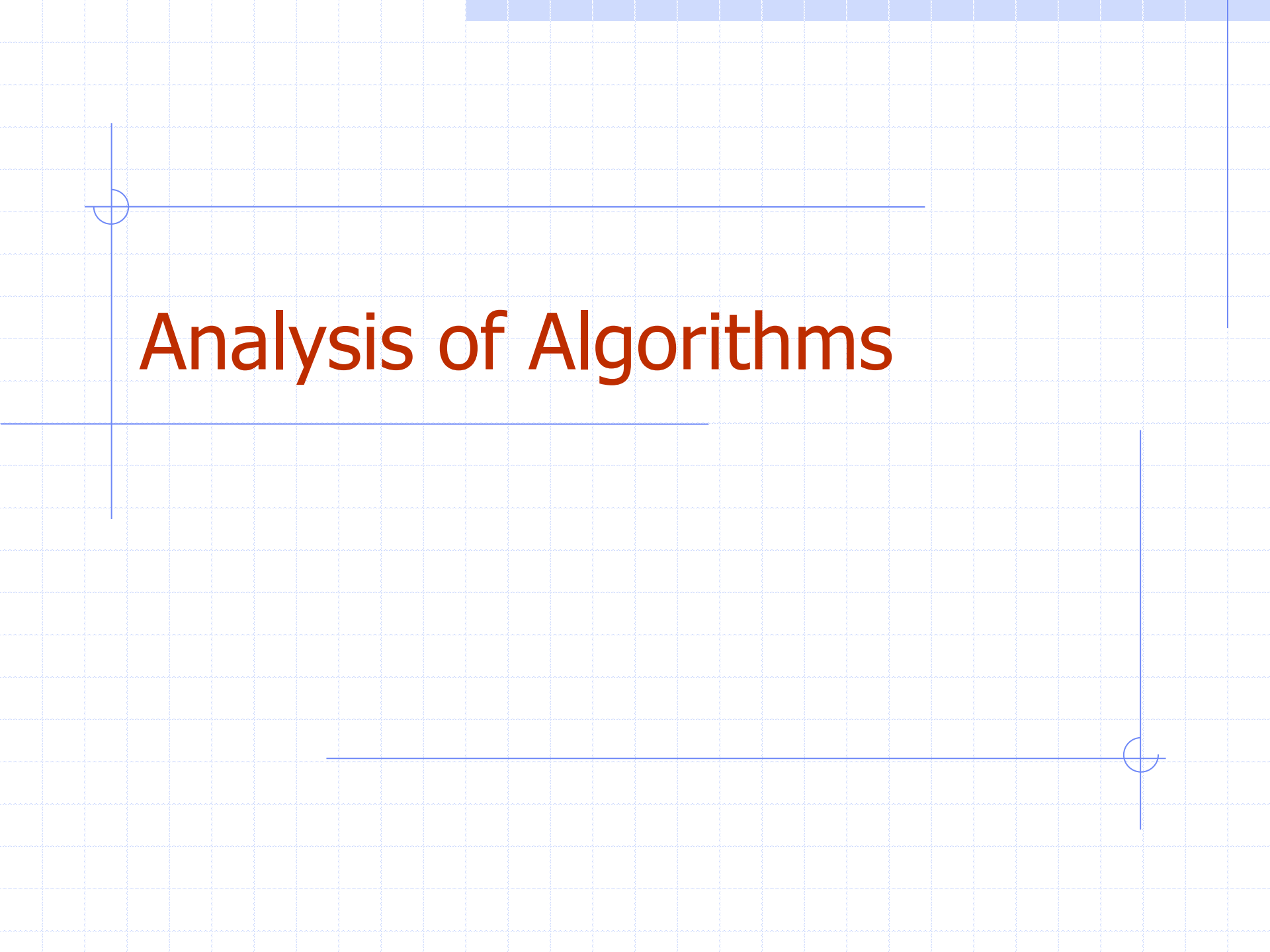
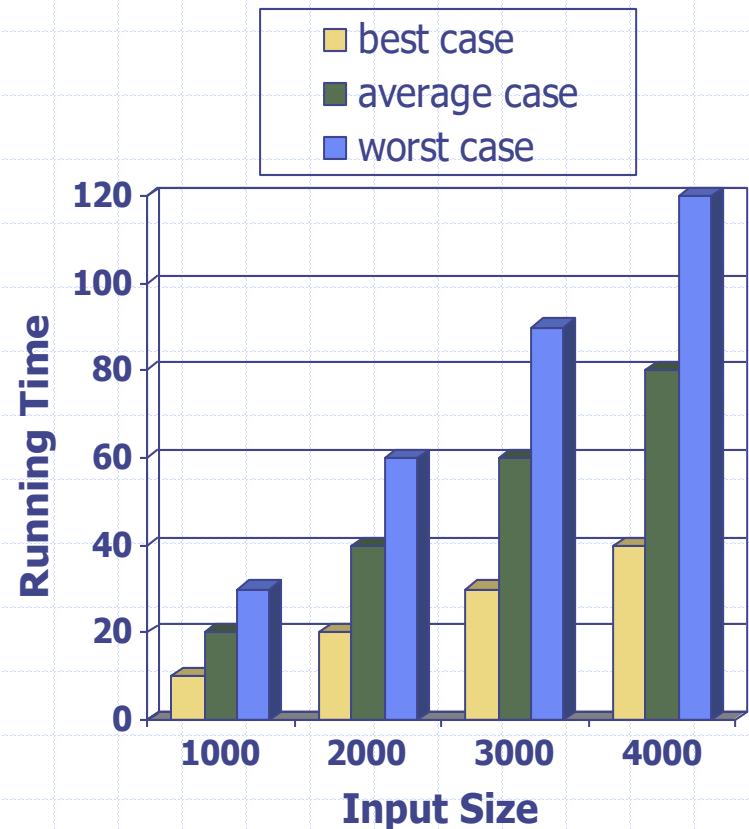


The background is a light blue grid. There are several blue lines and circles: a vertical line on the left, a horizontal line near the top, a horizontal line below the title, and a vertical line on the right. There are also small blue circles at the intersections of these lines.

Analysis of Algorithms

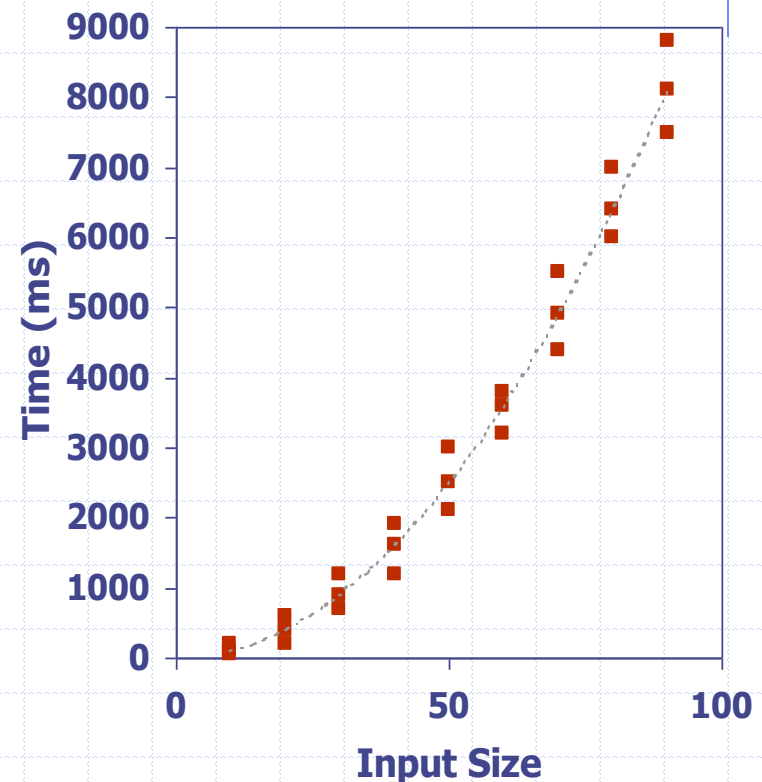
Analysis Framework

- ◆ Most algorithms transform input objects into output objects.
- ◆ The running time of an algorithm typically grows with the input size.
- ◆ Average case time is often difficult to determine.
- ◆ We focus on the worst case running time.
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics



Analysis Framework

- ◆ Time Efficiency
- ◆ Space efficiency
- ◆ Write a program implementing the algorithm with inputs of varying size and composition
- ◆ Use a method like `System.currentTimeMillis()` to get an accurate measure of the actual running time
- ◆ Plot the results



Time Efficiency

- ◆ It is necessary to implement the algorithm, which may be difficult
- ◆ Results may not be indicative of the running time on other inputs not included in the experiment.
- ◆ In order to compare two algorithms, the same hardware and software environments must be used

Theoretical Analysis for Time Efficiency

- ◆ Uses a high-level description of the algorithm instead of an implementation
- ◆ Characterizes running time as a function of the input size, n .
- ◆ Takes into account all possible inputs
- ◆ Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Pseudocode (§ 1.1)

- ◆ High-level description of an algorithm
- ◆ More structured than English prose
- ◆ Less detailed than a program
- ◆ Preferred notation for describing algorithms
- ◆ Hides program design issues

Example: find max element of an array

Algorithm *arrayMax*(*A*, *n*)

Input array *A* of *n* integers

Output maximum element of *A*

currentMax $\leftarrow A[0]$

for *i* $\leftarrow 1$ **to** *n* $- 1$ **do**

if *A*[*i*] > *currentMax* **then**

currentMax $\leftarrow A[i]$

return *currentMax*

Pseudocode Details

◆ Control flow

- **if ... then ... [else ...]**
- **while ... do ...**
- **repeat ... until ...**
- **for ... do ...**
- Indentation replaces braces

◆ Method declaration

Algorithm *method* (*arg* [, *arg*...])

Input ...

Output ...

◆ Method call

var.method (*arg* [, *arg*...])

◆ Return value

return *expression*

◆ Expressions

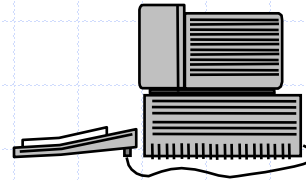
← Assignment
(like = in Java)

= Equality testing
(like == in Java)

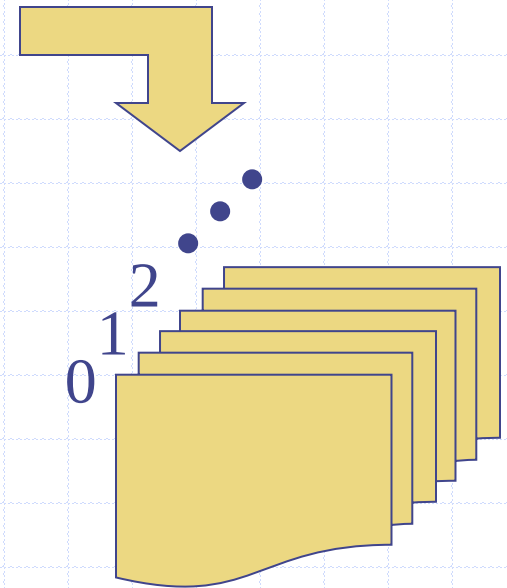
*n*² Superscripts and other
mathematical
formatting allowed

The Random Access Machine (RAM) Model

- ◆ A CPU



- ◆ An potentially unbounded bank of **memory** cells, each of which can hold an arbitrary number or character



- ◆ Memory cells are numbered and accessing any cell in memory takes unit time.

Primitive Operations

- ◆ Basic computations performed by an algorithm
- ◆ Identifiable in pseudocode
- ◆ Largely independent from the programming language
- ◆ Exact definition not important
- ◆ Assumed to take a constant amount of time in the RAM model

◆ Examples:

- Evaluating an expression
- Assigning a value to a variable
- Indexing into an array
- Calling a method
- Returning from a method

Counting Primitive Operations

- ◆ By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm <i>arrayMax</i> (<i>A</i> , <i>n</i>)	# operations
<i>currentMax</i> $\leftarrow$ <i>A</i> [0]	2
for <i>i</i> $\leftarrow$ 1 to <i>n</i> - 1 do	$2 + n$
if <i>A</i> [<i>i</i>] > <i>currentMax</i> then	$2(n - 1)$
<i>currentMax</i> $\leftarrow$ <i>A</i> [<i>i</i>]	$2(n - 1)$
{ increment counter <i>i</i> }	$2(n - 1)$
return <i>currentMax</i>	1
Total	$7n - 1$

Estimating Running Time

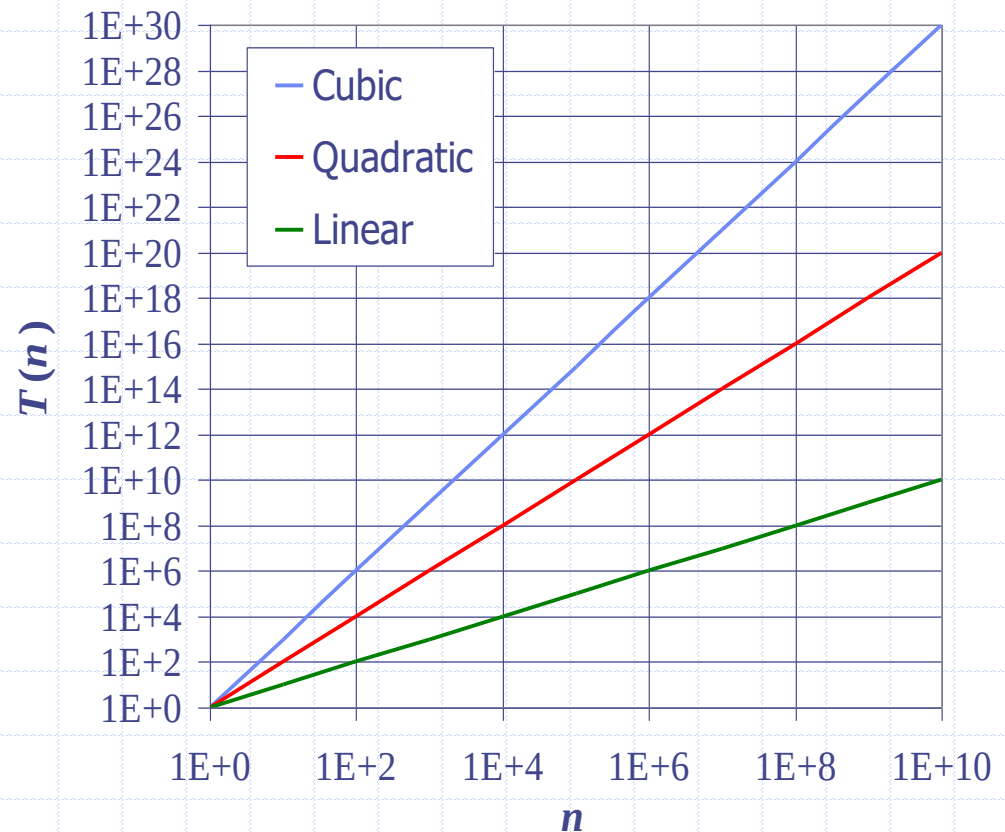
- ◆ Algorithm *arrayMax* executes $7n - 1$ primitive operations in the worst case. Define:
 - a = Time taken by the fastest primitive operation
 - b = Time taken by the slowest primitive operation
- ◆ Let $T(n)$ be worst-case time of *arrayMax*. Then
$$a(7n - 1) \leq T(n) \leq b(7n - 1)$$
- ◆ Hence, the running time $T(n)$ is bounded by two linear functions

Growth Rate of Running Time

- ◆ Changing the hardware/ software environment
 - Affects $T(n)$ by a constant factor, but
 - Does not alter the growth rate of $T(n)$
- ◆ The linear growth rate of the running time $T(n)$ is an intrinsic property of algorithm *arrayMax*

Growth Rates

- ◆ Growth rates of functions:
 - Linear $\approx n$
 - Quadratic $\approx n^2$
 - Cubic $\approx n^3$
- ◆ In a log-log chart, the slope of the line corresponds to the growth rate of the function

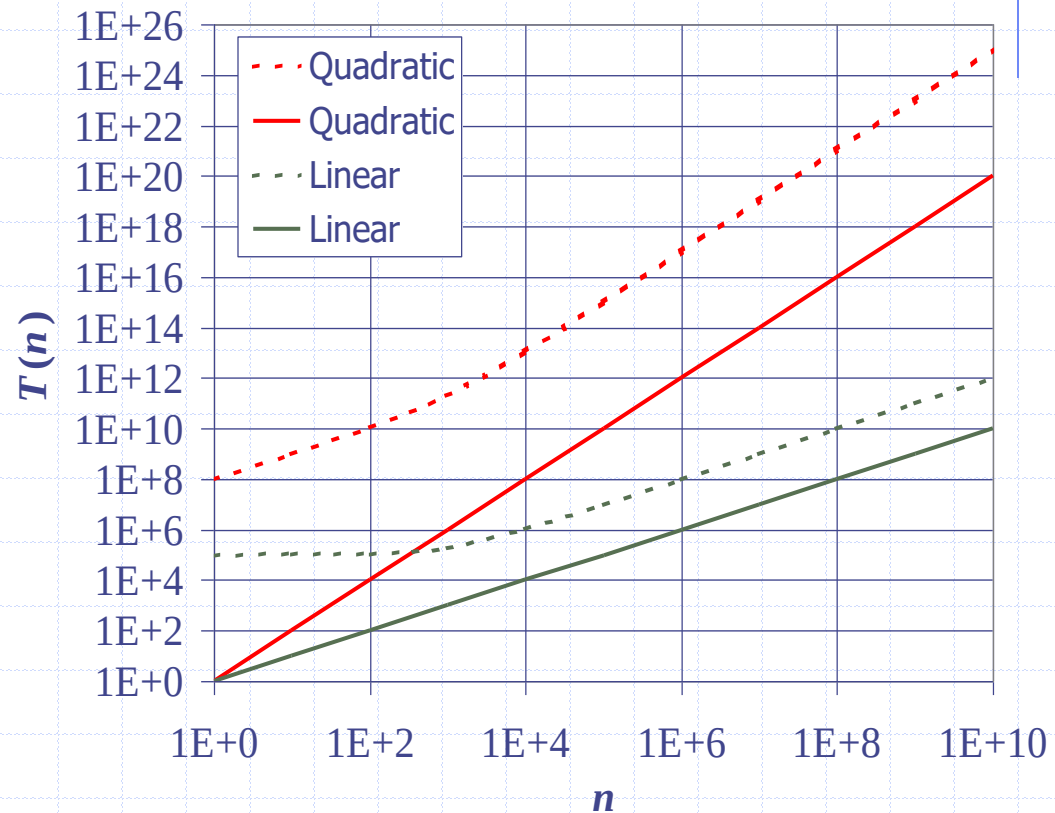


Constant Factors

- ◆ The growth rate is not affected by
 - constant factors
 - or
 - lower-order terms

- ◆ Examples

- $10^2n + 10^5$ is a linear function
- $10^5n^2 + 10^8n$ is a quadratic function



Worst-Case, Best-Case, and Average-Case Efficiencies

Algorithm *SequentialSearch*(A, n, K)

$i \leftarrow 0$

while $i < n$ *and* $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

Worst-Case, Best-Case, and Average-Case Efficiencies

- ◆ The running time of this algorithm can be quite different for the same list size n
- ◆ Worst-case efficiency of an algorithm is its efficiency for the algorithm runs the longest among all possible inputs of that size n .
- ◆ The worst-case analysis provides very important information about an algorithm's efficiency by bounding its running time from above.
- ◆ In other words, it guarantees that for any instance of size n , the running time will not exceed it.
- ◆ From the *SequentialSearch* algorithm the worse-case time is $T(n) = n$

Worst-Case, Best-Case, and Average-Case Efficiencies

- ◆ Best-case efficiency of an algorithm is its efficiency for the algorithm runs the fastest among all possible inputs of that size n .
- ◆ First, determine the kind of input for which the #of operations will be the smallest among all possible inputs of size n .
- ◆ Then, determine #of operations according that kind of input.
- ◆ From the *SequentialSearch* algorithm the best-case time $T(n) = 1$

Worst-Case, Best-Case, and Average-Case Efficiencies

- ◆ Average-case efficiency yield the necessary information an algorithm's behavior on a "typical" or "random" input.
- ◆ From the *SequentialSearch* algorithm, the probability of a successful search is p ($0 \leq p \leq 1$), the probability of the first math occurring in the i th position of the list is p/n for every i , the number of comparisons is n with probability of such a search being $(1-p)$.

Worst-Case, Best-Case, and Average-Case Efficiencies

◆ Therefore,

$$\begin{aligned}T_{\text{avg}}(n) &= [1 * p/n + 2 * p/n + \dots + i * p/n + \dots + n * p/n] + n * (1-p) \\&= p/n [1 + 2 + \dots + i + \dots + n] + n(1-p) \\&= p/n * n(n+1)/2 + n(1-p) = p(n-1)/2 + n(1-p)\end{aligned}$$

- ◆ If the search is must be successful ($p = 1$) the average = $(n+1)/2$
- ◆ If the search is must be unsuccessful ($p = 0$) the average = n

Worst-Case, Best-Case, and Average-Case Efficiencies

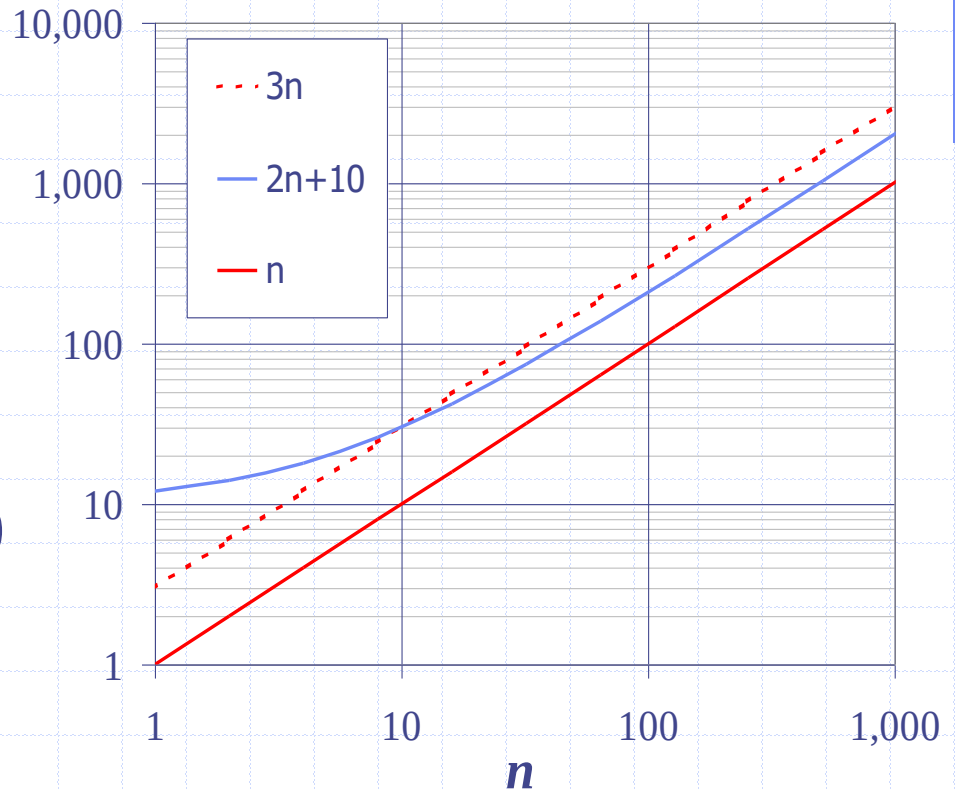
- ◆ A probability distribution of inputs needs to be obtained or assumed so that the expected value of the basic #of operations can be derived.
- ◆ Probability assumption is usually difficult to verify, but average-case efficiency information is needed. There are many algorithms for which the average-case efficiency is much better than the worst-case efficiency.

Asymptotic Notation (Big-Oh Notation)

◆ Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that $f(n) \leq cg(n)$ for $n \geq n_0$

◆ Example: $2n + 10$ is $O(n)$

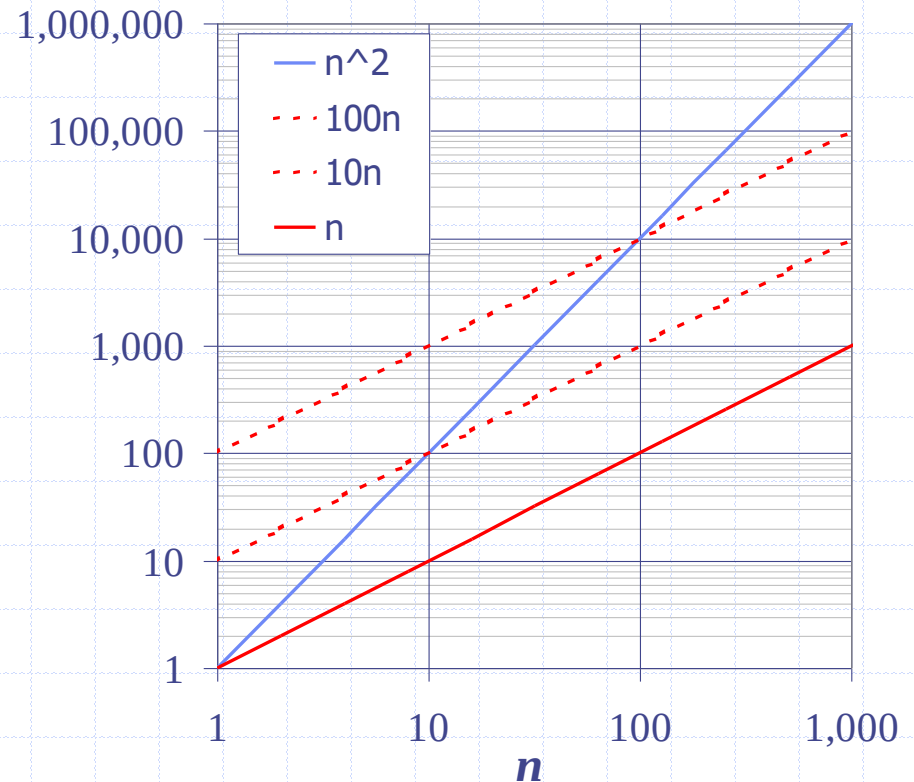
- $2n + 10 \leq cn$
- $(c - 2)n \geq 10$
- $n \geq 10/(c - 2)$
- Pick $c = 3$ and $n_0 = 10$



Big-Oh Example

◆ Example: the function n^2 is not $O(n)$

- $n^2 \leq cn$
- $n \leq c$
- The above inequality cannot be satisfied since c must be a constant



More Big-Oh Examples

- $7n-2$

$7n-2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n-2 \leq c \cdot n$ for $n \geq n_0$

this is true for $c = 7$ and $n_0 = 1$

- $3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for $n \geq n_0$

this is true for $c = 4$ and $n_0 = 21$

- $3 \log n + \log \log n$

$3 \log n + \log \log n$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + \log \log n \leq c \cdot \log n$ for $n \geq n_0$

this is true for $c = 4$ and $n_0 = 2$

Big-Oh and Growth Rate

- ◆ The big-Oh notation gives an upper bound on the growth rate of a function
- ◆ The statement “ $f(n)$ is $O(g(n))$ ” means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- ◆ We can use the big-Oh notation to rank functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Big-Oh Rules

- ◆ If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,
 1. Drop lower-order terms
 2. Drop constant factors
- ◆ Use the smallest possible class of functions
 - Say " $2n$ is $O(n)$ " instead of " $2n$ is $O(n^2)$ "
- ◆ Use the simplest expression of the class
 - Say " $3n + 5$ is $O(n)$ " instead of " $3n + 5$ is $O(3n)$ "

Asymptotic Algorithm Analysis

- ◆ The asymptotic analysis of an algorithm determines the running time in big-Oh notation
- ◆ To perform the asymptotic analysis
 - We find the worst-case number of primitive operations executed as a function of the input size
 - We express this function with big-Oh notation
- ◆ Example:
 - We determine that algorithm *arrayMax* executes at most $7n - 1$ primitive operations
 - We say that algorithm *arrayMax* “runs in $O(n)$ time”
- ◆ Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Math you need to Review

- ◆ Summations
- ◆ Logarithms and Exponents
- ◆ Proof techniques
- ◆ Basic probability

◆ **properties of logarithms:**

$$\log_b(xy) = \log_b x + \log_b y$$

$$\log_b (x/y) = \log_b x - \log_b y$$

$$\log_b xa = a \log_b x$$

$$\log_b a = \log_x a / \log_x b$$

◆ **properties of exponentials:**

$$a^{(b+c)} = a^b a^c$$

$$a^{bc} = (a^b)^c$$

$$a^b / a^c = a^{(b-c)}$$

$$b = a^{\log_a b}$$

$$b^c = a^{c \cdot \log_a b}$$

Relatives of Big-Oh

◆ big-Omega

- $f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

◆ big-Theta

- $f(n)$ is $\Theta(g(n))$ if there are constants $c' > 0$ and $c'' > 0$ and an integer constant $n_0 \geq 1$ such that $c' \cdot g(n) \leq f(n) \leq c'' \cdot g(n)$ for $n \geq n_0$

◆ little-oh

- $f(n)$ is $o(g(n))$ if, for any constant $c > 0$, there is an integer constant $n_0 \geq 0$ such that $f(n) \leq c \cdot g(n)$ for $n \geq n_0$

◆ little-omega

- $f(n)$ is $\omega(g(n))$ if, for any constant $c > 0$, there is an integer constant $n_0 \geq 0$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

Intuition for Asymptotic Notation

Big-Oh

- $f(n)$ is $O(g(n))$ if $f(n)$ is asymptotically **less than or equal** to $g(n)$

big-Omega

- $f(n)$ is $\Omega(g(n))$ if $f(n)$ is asymptotically **greater than or equal** to $g(n)$

big-Theta

- $f(n)$ is $\Theta(g(n))$ if $f(n)$ is asymptotically **equal** to $g(n)$

little-oh

- $f(n)$ is $o(g(n))$ if $f(n)$ is asymptotically **strictly less** than $g(n)$

little-omega

- $f(n)$ is $\omega(g(n))$ if $f(n)$ is asymptotically **strictly greater** than $g(n)$

Examples

- $5n^2$ is $\Omega(n^2)$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

let $c = 5$ and $n_0 = 1$

- $5n^2$ is $\Omega(n)$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

let $c = 1$ and $n_0 = 1$

- $5n^2$ is $\omega(n)$

$f(n)$ is $\omega(g(n))$ if, for any constant $c > 0$, there is an integer constant $n_0 \geq 0$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

need $5n_0^2 \geq c \cdot n_0 \rightarrow$ given c , the n_0 that satisfies this is $n_0 \geq c/5 \geq 0$

Examples

■ $3n^2 - 100n + 6 = O(n^2)$?

Yes Because $3n^2 > 3n^2 - 100n + 6$

■ $3n^2 - 100n + 6 = O(n^3)$?

Yes Because $0.01n^3 > 3n^2 - 100n + 6$

■ $3n^2 - 100n + 6 = O(n)$?

≠ $O(n)$ Because $c \cdot n < 3n^2$ when $n > c$

■ $3n^2 - 100n + 6 = \Omega(n^2)$?

Yes Because $2.99n^2 < 3n^2 - 100n + 6$

■ $3n^2 - 100n + 6 = \Omega(n^3)$?

≠ $\Omega(n)$ Because $3n^2 - 100n + 6 < 3n^3$

■ $3n^2 - 100n + 6 = \Omega(n)$?

Yes Because $10^{100}n < 3n^2 - 100n + 6$

Using Limits for Comparing Orders of Growth

- ◆ $\lim_{n \rightarrow \infty} g(n)/f(n) = 0$ implies that $g(n)$ has a smaller order of growth than $f(n)$
- ◆ $\lim_{n \rightarrow \infty} g(n)/f(n) = c$ implies that $g(n)$ has a same order of growth as $f(n)$
- ◆ $\lim_{n \rightarrow \infty} g(n)/f(n) = \infty$ implies that $g(n)$ has a larger order of growth than $f(n)$

- ◆ Moreover according to L'Hopital's rule

$$\lim_{n \rightarrow \infty} g(n)/f(n) = \lim_{n \rightarrow \infty} g'(n)/f'(n)$$

- ◆ And Stirling's formula

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad \text{for very large values of } n$$

Using Limits for Comparing Orders of Growth

- ◆ Compare orders of growth of $(1/2)n(n-1)$ and n^2
- ◆ Compare orders of growth of $\log(n)$ and $n^{(1/2)}$
- ◆ Compare orders of growth of $n!$ and 2^n

Properties of Logarithms

- ◆ Recall, $c^{\log_c x} = x$
- ◆ Asymptotically, the base of the *log* doesn't matter
 - $\log_b a = \log_c a / \log_c b$
 - Thus, $\log_2 n = (1 / \log_{100} 2) * \log_{100} n$, $1 / \log_{100} 2 = 6.643$ (constant)
- ◆ Asymptotically, polynomial function of n doesn't matter
 - $\log(n^{473} + n^2 + n + 96) = O(\log n)$
 - Since, $n^{473} + n^2 + n + 96 = O(n^{473})$, and $\log(n^{473}) = 473 * \log n$

Analysis of Non-recursive Algorithms

```
Algorithm MaxElement (A[0...n-1])  
  maxvalue  $\leftarrow A[0]$   
  for  $i \leftarrow 0$  to  $n - 1$  do  
    if  $A[i] > \text{maxvalue}$   
      maxvalue  $\leftarrow A[i]$   
  return maxvalue
```

1

- ◆ $T(n)$ = the number of times is executed in an array of size n
- ◆ Algorithm makes **one** comparison on each execution of the loop which is repeated for each value of the i within bound of 1 and $n-1$

$$T(n) = \sum_{i=1}^{n-1} 1$$

$$T(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

Analysis of Non-recursive Algorithms

```
Algorithm UniqueElements (A[0...n-1])  
  for  $i \leftarrow 0$  to  $n - 2$  do  
    for  $j \leftarrow i + 1$  to  $n - 1$  do  
      if  $A[i] = A[j]$  return false  
  return true
```

1

◆ Innermost loop contains a single operation ($A[i] = A[j]$)

$$\begin{aligned} T_{\text{worst}}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) \\ &= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \\ &= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2) \end{aligned}$$

Analysis of Non-recursive Algorithms

```
Algorithm MatrixMultiplication (A[0...n-1][0...n-1], B[0...n-1][0...n-1])  
  for  $i \leftarrow 0$  to  $n - 1$  do  
    for  $j \leftarrow 0$  to  $n - 1$  do  
       $C[i,j] \leftarrow 0$   
      for  $k \leftarrow 0$  to  $n-1$  do  
         $C[i,j] \leftarrow C[i,j] + A[i,k]*B[k,j]$  1  
  return  $C$ 
```

- ◆ There is just one multiplication executed on each repetition of algorithm's innermost loop.

$$\sum_{k=0}^{n-1} 1 \quad T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 \quad T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{j=0}^{n-1} n^2 = n^3$$

Analysis of Non-recursive Algorithms

Algorithm *Binary* (n)

count $\leftarrow 1$

while $n > 1$ **do**

count $\leftarrow$ *count* + 1

n $\leftarrow n/2$

return *count*

- ◆ Most frequently executed operation here is not inside the **while** loop but rather the comparison $n > 1$.
- ◆ We have to compute the number of times the loop is executed.
- ◆ Value of n is about halved on each repetition of the loop.
- ◆ The answer should be about $\log_2 n$

Plan for Analyzing Efficiency of Non-recursive Algorithm

- ◆ Decide on a parameter (s) indicating an input's size.
- ◆ Identify the algorithm's basic operation (inner most loop).
- ◆ Check that the number of times the basic operations is executed depended only on the size of an input.
- ◆ Set up a sum expressing the number of times the algorithm's basic operation is executed.
- ◆ Using standard formulas and rules of sum manipulation, either find a closed form formula for the count.

Analysis of Recursive Algorithms

Algorithm *Factorial*(n) $// n! = 1 * \dots * (n-1) * n = (n-1)! * n$ for $n \geq 1$
if $n = 0$ **return** 1
else return *Factorial*($n-1$) * n

- ◆ The basic operation is multiplication, whose number of executions = $T(n)$.
- ◆ The function $F(n)$ is computed according to the formula
$$F(n) = F(n-1) * n \text{ for } n > 0$$
- ◆ The number of multiplication $T(n)$ needed to compute it must satisfy the equality

$$T(n) = T(n-1) + 1 \text{ for } n > 0$$

↑
To compute
 $F(n-1)$

↑
To multiply
 $F(n-1)$ by n

Analysis of Recursive Algorithms

- ◆ Our goal is to solve the recurrence relation (recurrence)
 $T(n) = T(n-1) + 1$
- ◆ Condition that makes the algorithm stop: **if** $n = 0$ **return** 1
- ◆ This implies 1. the calls stop when $n = 0$, 2. the smallest of n is 0. Therefore;

$$T(n) = T(n-1) + 1 \text{ for } n > 0$$

$$T(0) = 0 \text{ // no multiplication when } n = 0$$

$$F(n) = F(n-1) * n \text{ for } n > 0$$

$$F(0) = 1$$

Analysis of Recursive Algorithms

◆ Method of Backward Substitutions

$$T(n) = T(n-1) + 1$$

$$= [T(n-2) + 1] + 1 = T(n-2) + 2$$

$$= [T(n-3) + 2] + 1 = T(n-3) + 3$$

$$\vdots$$

$$T(n) = T(n-i) + i$$

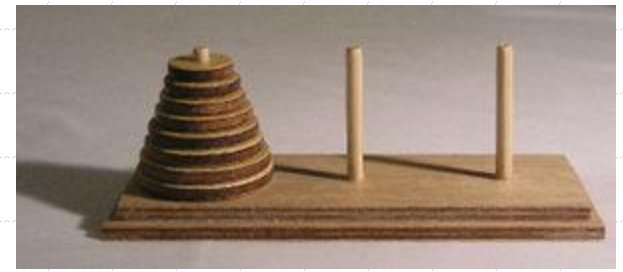
$$\text{substitute } T(n-1) = T(n-2) + 1$$

$$\text{substitute } T(n-2) = T(n-3) + 1$$

- ◆ We take advantage of the initial condition given that it is specified $n=0$, by substitute $i = n$

$$T(n) = T(n-1) + 1 \dots = T(n-i) + i = \dots = T(n-n) + n = n$$

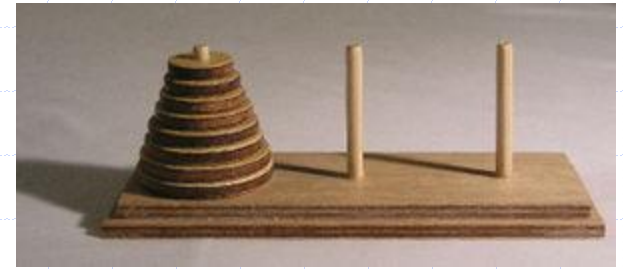
Analysis of Recursive Algorithms



- ◆ Tower of Hanoi (<http://www.mazeworks.com/hanoi/>) and (http://en.wikipedia.org/wiki/Tower_of_Hanoi)
- ◆ This puzzle contains n disks of different sizes and three pegs. First all the disks are on the first peg in order of size, the largest on the bottom and the smallest on top.
- ◆ The goal is to move all disks to the third pegs using second one as an auxiliary, if necessary.
- ◆ Only one disk can be moved at a time, and it is forbidden to place a larger disk on top of a smaller one.

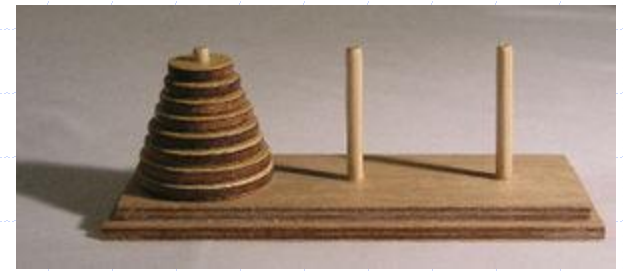


Analysis of Recursive Algorithms



- ◆ To move $n > 1$ disks from peg 1 to peg 3, we first move recursively $n-1$ disks from peg 1 to peg 2, then move the largest disk directly from peg 1 to peg 3 and move recursively $n-1$ disks from peg 2 to peg 3.
- ◆ If $n = 1$, move the single disk from peg 1 to peg 3.
- ◆ The number of disks n is the choice for the input size and moving one disk is the algorithm's basic operation.
- ◆ The number of move $T(n)$ follow the recurrence equation:
$$T(n) = T(n-1) + 1 + T(n-1) \text{ for } n > 1 \text{ where}$$
$$T(1) = 1 \text{ // initial condition}$$

Analysis of Recursive Algorithms



$$T(n) = 2T(n-1) + 1$$

$$= 2[2T(n-2) + 1] + 1 =$$

$$= 2^2T(n-3) + 1 + 2 + 1 =$$

The next substitute would be

$$\text{sub } T(n-1) = 2T(n-2) + 1$$

$$2^2T(n-2) + 2 + 1 \quad \text{sub } T(n-2) = 2T(n-3) + 1$$

$$2^3T(n-3) + 2^2 + 2 + 1$$

$$2^4T(n-4) + 2^3 + 2^2 + 2 + 1$$

$$T(n) = 2^i T(n-i) + 2^{i-1} + 2^{i-2} + \dots + 2 + 1 = 2^i T(n-i) + 2^i - 1$$

- ◆ Initial condition is for $n=1$, which is achieved for $i = n-1$, therefore we get following formula for solution

$$T(n) = 2^{n-1} T(n-(n-1)) + 2^{n-1} - 1$$

$$= 2^{n-1} T(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1$$

Analysis of Recursive Algorithms

```
Algorithm BinRec(n)
  if  $n = 1$  return 1
  else return BinRec(n/2)+1
```

- ◆ From the initial condition, the time efficiency is the number of addition made by the algorithm = $T(n)$.
- ◆ Then number of addition made in computing $\text{BinRec}(n/2)$ is $T(n/2)$ plus one more addition to increase the returned valued by 1. Therefore:

$$T(n) = T(n/2) + 1 \text{ for } n > 1$$

$$T(1) = 0 \text{ //the recursive calls end when } n = 1$$

Analysis of Recursive Algorithms

◆ Let $n=2^k$ then $T(2^k) = T(2^{k-1})+1$ for $n>0$ where $T(2^0) = 0$

◆ Backward Substitution:

$$T(2^k) = T(2^{k-1})+1$$

$$\text{substitute } T(2^{k-1}) = T(2^{k-2})+1$$

$$= [T(2^{k-2}) + 1] + 1 = T(2^{k-2}) + 2$$

$$\text{substitute } T(2^{k-2}) = T(2^{k-3}) + 1$$

$$= [T(2^{k-3}) + 1] + 2 = T(2^{k-3}) + 3$$

$$\vdots$$

$$= T(2^{k-i}) + i$$

$$= T(2^{k-k}) + k \quad // \text{ let } i = k$$

$$T(2^k) = T(2^0) + k = k$$

◆ Since $n=2^k$, which also means $k = \log_2 n$ therefore:

$$T(n) = \log_2 n = \Theta(\log_2 n)$$

Plan for Analyzing Efficiency of Recursive Algorithm

- ◆ Decide on a parameter (s) indicating an input's size.
- ◆ Identify the algorithm's basic operation (inner most loop).
- ◆ Check that the number of times the basic operations is executed depended only on the size of an input.
- ◆ Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
- ◆ Solve the recurrence or ascertain the order of growth of its solution.

Exercises on Recursive Algorithms

◆ $T(n) = 3T(n/4) + n$

◆ $T(n) = 2T(n/2) + n^2$

Tail Recursive

◆ function recsum(x) {
 if (x==1)
 { return x; }
 else
 { return x + recsum(x-1); }
}

◆ function recsum2(x, running_total=0) {
 if (x==0)
 { return running_total; }
 else
 { return recsum2(x-1, running_total+x); }
}